Oliver Krauss

Modern Code - Input, Output & Validation

ModernCode | MTD.ba VZ SS25

Hagenberg 01.12.2025

HAGENBERG | LINZ | STEYR | WELS

FH OBERÖSTERREICH

UNIVERSITY OF APPLIED SCIENCES UPPER AUSTRIA

Input, Output & Validation

- **Why I/O matters**: Programs need to interact with users
- **Connection to previous topics**: Methods + strings + file I/O → now console I/O
- **Real-world relevance**: Every interactive program needs input validation
- **The problem**: User input is unpredictable and must be validated
- **The solution**: Read carefully, validate thoroughly, handle errors gracefully

Review: Output Methods We Know

- **IO.println()**: What we use in this course (simple wrapper)
- **System.out.println()**: Standard Java output to console
- **System.out.print()**: Same but without automatic newline
- **printf() and String.format()**: Covered in Session 8 (Strings)

```
1   IO.println("Hello World!");           // Course wrapper
2   System.out.println("Hello World!");   // Standard Java
3   System.out.print("No newline");       // Stays on same line
```

Output Streams: out vs err

```
System.out.println("Normal output");    // Standard output stream
System.err.println("Error message!");   // Error stream (often red)

// Example: Validation feedback
int age = -5;
if (age < 0) {
    System.err.println("ERROR: Age cannot be negative");
    System.out.println("Please enter a valid age.");
}
```

When to use each:

- `System.out` - Normal program output, user messages
- `System.err` - Error messages, warnings, diagnostic info

Formatted Output: Quick Refresher

```
String name = "Alice";
int score = 95;
double accuracy = 0.875;

// printf() - formatted output (Session 8)
System.out.printf("Player: %s, Score: %d, Accuracy: %.1f%%%n",
                  name, score, accuracy * 100);
// Output: Player: Alice, Score: 95, Accuracy: 87.5%
```

Key format specifiers (from Session 8):

- `%s` - String, `%d` - integer, `%f` - floating-point
- `%.2f` - 2 decimal places, `%n` - platform-independent newline

Introduction to Scanner

- **What is Scanner?** Reads formatted input from various sources
- **Why we need it**: `System.in` provides raw bytes, Scanner parses them
- **Creating Scanner**: Connect to console input stream (we already did this with files)

```
1   import java.util.Scanner;
2
3   Scanner scanner = new Scanner(System.in);
4
5   // Now we can read user input:
6   String input = scanner.nextLine();
7   IO.println("You entered: " + input);
```

Important: Scanner can read from console, files, strings, etc.

Reading Different Data Types

```
String word = scanner.next();
String line = scanner.nextLine();
int number = scanner.nextInt();
double decimal = scanner.nextDouble();
boolean flag = scanner.nextBoolean();
```

- `next()` - reads until whitespace (one word at a time)
- `nextLine()` - reads until newline (entire line including spaces)
- `nextInt()` - reads and converts to integer
- `nextDouble()` - reads and converts to double
- `nextBoolean()` - reads and converts to boolean
- These methods will crash if input doesn't match expected type!

Checking Before Reading

```
1   boolean hasInt = scanner.hasNextInt();
2   boolean hasDouble = scanner.hasNextDouble();
3   boolean hasLine = scanner.hasNextLine();
4   boolean hasNext = scanner.hasNext();
```

What each check does:

- `hasNextInt()` - checks if next token can be parsed as integer
- `hasNextDouble()` - checks if next token can be parsed as double
- `hasNextLine()` - checks if there's a complete line available
- `hasNext()` - checks if there's any token left to read

Safe Reading Pattern

```
1   if (scanner.hasNextInt()) {
2       int number = scanner.nextInt();
3       IO.println("Got number: " + number);
4   } else {
5       String text = scanner.next();
6       IO.println("Not a number: " + text);
7   }
```

Why check first?

- Prevents crashes from wrong data types
- Allows graceful handling of invalid input
- Better user experience with clear error messages

Scanner vs IO Helper Methods

Two approaches - we'll use both:

Raw Scanner (Standard Java):

```
1   Scanner sc = new Scanner(
        System.in);
2   IO.println("Enter age:");
3   int age = sc.nextInt();
```

IO Helper (Course wrapper):

```
1   IO.println("Enter age:");
2   String input = IO.readLn();
3   int age = Integer.parseInt(
        input);
```

When to use each:

- **IO helpers**: Simpler for basic input, still need to parse manually
- **Raw Scanner**: More control, built-in parsing, real-world code

The nextLine() Pitfall

```
1   Scanner scanner = new Scanner(System.in);
2   IO.println("Enter your age:");
3   int age = scanner.nextInt();
4   IO.println("Enter your name:");
5   String name = scanner.nextLine();
6
7   IO.println("Age: " + age);  // Works
8   IO.println("Name: " + name);// Empty string!
```

- User types: `25[ENTER]` → creates `"25\n"`
- `nextInt()` reads `25`, leaves `\n` in buffer
- `nextLine()` reads the leftover `\n` → empty string

Fixing the nextLine() Problem I

Solution 1: Extra nextLine() to consume newline

```
1   Scanner scanner = new Scanner(System.in);
2
3   IO.println("Enter your age:");
4   int age = scanner.nextInt();
5   scanner.nextLine();  // Consume leftover newline
6
7   IO.println("Enter your name:");
8   String name = scanner.nextLine();  // Now works correctly
```

Fixing the nextLine() Problem II

Solution 2: Always use nextLine() + parse manually

```
1  Scanner scanner = new Scanner(System.in);
2
3  IO.println("Enter your age:");
4  String ageStr = scanner.nextLine();
5  int age = Integer.parseInt(ageStr);  // Parse string to int
6
7  IO.println("Enter your name:");
8  String name = scanner.nextLine();  // Consistent approach
```

Fixing the nextLine() Problem III

Solution 3: Use IO helpers (they handle this)

```
1   IO.println("Enter your age:");
2   String ageStr = IO.readLn();
3   int age = Integer.parseInt(ageStr);
4
5   IO.println("Enter your name:");
6   String name = IO.readLn();
```

Recommendation: Use Solution 2 or 3 for cleaner code

The Parsing Challenge

Problem: User input comes as strings, but we need other types

Example scenario:

- User types: `"25"` (string)
- Program needs: `int` value 25
- Challenge: Convert safely without crashing

What can go wrong?

- User types `"abc"` instead of `"25"`
- User types `" 25 "` (with spaces)
- User types `"25.5"` when we want integer

Solution 1: Basic Parsing I

```
1  // Convert strings to different types
2  String ageStr = "25";
3  int age = Integer.parseInt(ageStr);
4
5  String priceStr = "19.99";
6  double price = Double.parseDouble(priceStr);
```

Solution 1: Basic Parsing II

```
1   String flagStr = "true";
2   boolean flag = Boolean.parseBoolean(flagStr);
3
4   String bigStr = "1000000000";
5   long big = Long.parseLong(bigStr);
```

Pattern: `WrapperClass.parseType(String)`

Problem: These methods crash on invalid input!

Solution 2: Handle Crashes with try-catch

```
1  String input = "abc";
2  try {
3      int number = Integer.parseInt(input);
4      IO.println("Valid number: " + number);
5  } catch (NumberFormatException e) {
6      System.err.println("ERROR: '" + input + "' is not a number");
7  }
```

- Try to parse the string
- If it works: use the result
- If it fails: catch the exception and handle gracefully

Result: No crashes, clear error messages

Solution 3: Clean Input First

```
String input = " aBc ";

// Clean the input before parsing
String cleaned = input.trim();   // Remove leading+trailing spaces
cleaned = cleaned.toLowerCase(); // Normalize case

// now parse
```

- `trim()` - remove leading/trailing whitespace
- `toLowerCase()` - normalize case
- `replace()` - remove unwanted characters

Best practice: Clean first, then parse safely

What is Input Validation?

- **Definition**: Checking input meets requirements before using it
- **First line of defense**: Catch problems at the boundary
- **Types of validation**:
  - **Type validation**: Is it a number? A string? A boolean?
  - **Range validation**: Is $0 \leq age \leq 120$? Is $1 \leq choice \leq 5$?
  - **Format validation**: Does email contain @? Is phone 10 digits?
  - **Business rules**: Is username unique? Is password strong enough?

Key principle: Never trust user input!

Range Validation

```
1   IO.println("Enter your age (0-120):");
2   String ageStr = IO.readLn();
3   int age = Integer.parseInt(ageStr);
4
5   // Validate range
6   if (age < 0 || age > 120) {
7       System.err.println("ERROR: Age must be between 0 and 120");
8       System.err.println("Got: " + age);
9       System.out.println("Please enter a valid age.");
10  } else {
11      IO.println("Valid age: " + age);
12  }
```

Format Validation with Regex

```
1  // Simple email validation (basic pattern)
2  String emailPattern = "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$";
3
4  if (email.matches(emailPattern)) {
5      IO.println("Valid email format: " + email);
6  } else {
7      System.err.println("ERROR: Invalid email format");
8  }
```

From Session 8: `matches()` checks string against regex pattern

Common format validations: Email, phone number, username, password

The Retry Pattern

- **Goal**: Keep asking until valid input received
- **Why do-while?** Must ask at least once
- **Loop structure**:
  1. Ask for input
  2. Validate input
  3. If invalid: show error, loop again
  4. If valid: break loop, continue program
- **Safety**: Add attempt limit to prevent infinite loops

Real-world use: Menu systems, user registration, game input

Simple Retry Loop Example

```
1   int number;
2   boolean valid = false;
3   do {
4       IO.println("Enter a number between 1 and 10:");
5       String input = IO.readLn();
6       number = Integer.parseInt(input);
7       if (number >= 1 && number <= 10) {
8           valid = true;  // Input is good, exit loop
9       } else {
10          System.err.println("Invalid! Must be between 1 and 10.");
11      }
12  } while (!valid);
```

Loops until `valid` becomes `true`

Retry with Attempt Limit

```
1  int number;
2  boolean valid = false;
3  int attempts = 0;
4  int maxAttempts = 3;
5
6  do {
7      attempts++;
8      ...
9  } while (!valid && attempts < maxAttempts);
```

Error Messages: Best Practices I

Bad error messages:

```
System.err.println("Error");
System.err.println("Invalid");
System.err.println("Bad input");
```

Good error messages:

```
System.err.println("Age must be 0-120");
System.err.println("Got: 'abc', expected: number");
System.err.println("Username: 3-15 chars only");
```

Error Messages: Best Practices II

Good error messages include:

- What's wrong: "Age cannot be negative"
- What's expected: "Must be between 0 and 120"
- What was received: "Got: -5"
- How to fix it: "Enter a positive number"

Defensive Programming

- **Definition**: Write code expecting the unexpected
- **Goal**: Robust, reliable programs even with bad input or edge cases
- **Connection to input validation**: Validation is defensive programming
- **Key principles**:
  - Validate all inputs before processing
  - Handle edge cases explicitly
  - Provide meaningful error messages
  - Fail fast and clearly when something goes wrong
- **Benefits**: Fewer crashes, easier debugging, better user experience

Input Validation Examples: Before

```
1   static int computeDamage(int baseDamage, double critMultiplier, int
        armor) {
2       int rawDamage = (int) Math.round(baseDamage * critMultiplier);
3       return Math.max(1, rawDamage - armor);
4   }
```

This method has no input validation and can produce unexpected results or crash with invalid inputs.

Input Validation Examples: Validate inputs first

```
1  static int computeDamage(int baseDamage, double crit, int armor) {
2    if (baseDamage < 0) {
3      throw new IllegalArgumentException("Base damage cannot be
       negative: " + baseDamage);
4    }
5    if (crit <= 0) {
6      throw new IllegalArgumentException("Critical multiplier must be
        positive: " + crit);
7    }
8    if (armor < 0) {
9      throw new IllegalArgumentException("Armor cannot be negative: "
        + armor);
10   }
```

Edge Case Handling

```
1   static int applyHealing(int currentHp, int healAmount, int maxHp) {
2     // Validate inputs first then ->
3     // Handle edge case: already at max health
4     if (currentHp >= maxHp) {
5         return currentHp;  // No healing needed
6     }
7     // Handle edge case: healing would exceed max
8     int newHp = currentHp + healAmount;
9     if (newHp > maxHp) {
10        return maxHp;  // Cap at maximum
11    }
12    return newHp;
13  }
```

Graceful Degradation

```
1  // Graceful degradation: handle errors without crashing
2  static int safeDivide(int numerator, int denominator) {
3    if (denominator == 0) {
4      IO.println("Warning: Division by zero attempted");
5      return 0;  // Return safe default value
6    }
7  }
```

Choose between graceful degradation (return safe defaults) or fail fast (throw exceptions) based on your application's needs.

Defensive Programming Patterns

- **Guard clauses**: Check invalid conditions first and return early
- **Default values**: Provide sensible fallbacks for edge cases
- **Input sanitization**: Clean and normalize data before processing
- **Boundary checking**: Always verify array indices and collection access
- **Resource management**: Ensure files, connections, etc. are properly closed
- **Error logging**: Record problems for debugging without exposing details to users

Guard Clauses: Before (Nested Ifs)

```
1   static String getPlayerStatus(int hp, int mp, boolean isAlive) {
2       if (isAlive) {
3           if (hp > 0) {
4               if (mp > 0) {
5                   return "Ready for battle";
6               } else {
7                   return "Low on mana";
8               }
9           } else {
10              return "Critical health";
11          }
12      } else {
13          return "Defeated";
14      }
15  }
```

Guard Clauses: After (Clean Pattern)

```
1  static String getPlayerStatus(int hp, int mp, boolean isAlive) {
2      if (!isAlive) return "Defeated";
3      if (hp <= 0) return "Critical health";
4      if (mp <= 0) return "Low on mana";
5      return "Ready for battle";
6  }
```

Benefits of guard clauses:

- Handle edge cases first
- Flatter, more readable code
- Easier to maintain and extend

When to Use Defensive Programming

- **Always use**:
  - Public methods that others will call
  - Methods that process user input
  - Methods that handle external data (files, network)
  - Critical business logic methods
- **Consider using**:
  - Private helper methods in complex classes
  - Methods that perform calculations with many parameters
  - Methods that could fail in unexpected ways
- **Balance**: Don’t over-validate simple, internal methods

Testing Defensive Code

- **Test invalid inputs**: Negative numbers, null values, out-of-range values
- **Test edge cases**: Zero values, maximum values, boundary conditions
- **Test error messages**: Ensure exceptions have meaningful information
- **Test graceful degradation**: Verify safe default values are returned
- **Test resource cleanup**: Ensure files and connections are properly closed
- **Use assertions**: Verify that validation catches bad inputs

Always test valid and invalid inputs

```
1   // Test valid inputs
2   assert computeDamage(10, 1.5, 2) == 13 : "Valid input failed";
3   // Test invalid inputs - should throw exceptions
4   try {
5     computeDamage(-5, 1.5, 2);
6     assert false : "Should have thrown exception negative damage";
7   } catch (IllegalArgumentException e) {
8     IO.println("âlJŞ Correctly rejected negative damage");
9   }
10  // Test edge cases
11  assert applyHealing(95, 10, 100) == 100 : "Healing cap at max HP";
12  assert applyHealing(100, 10, 100) == 100 : "No healing at max HP";
```

Common Input Mistakes

1. **Not validating input**
   - **Problem**: Crashes on invalid data
   - **Fix**: Always validate before using
2. **Poor error messages**
   - **Problem**: "Error" - what error?
   - **Fix**: Be specific: "Age must be 0-120, got: -5"
3. **Infinite retry loops**
   - **Problem**: Asking forever
   - **Fix**: Add attempt limit or exit option
4. **Scanner buffer issues**
   - **Problem**: Mixing nextInt() and nextLine()
   - **Fix**: Use IO helpers or nextLine() + parse

AI Support for Input Validation

AI can help with:

Generate validation patterns:

- "Create regex for username validation (letters, numbers, 3-15 chars)"
- "Generate email validation code in Java"
- "Create phone number validation for format XXX-XXX-XXXX"

Add defensive programming to existing code:

- "Add input validation to this method that calculates area"
- "Make this file reading code more robust with error handling"
- "Add null checks and range validation to this array processing method"

Suggest strategies:

- "Best way to validate age input in Java?"
- "Should I use Scanner or BufferedReader for console input?"
- "How to create a retry loop with attempt limit?"

Summary: Input, Output & Validation

Output Basics

- System.out vs System.err streams
- printf() for formatted output
- Platform-independent newlines (%n)

Scanner Fundamentals

- Reading different types (next, nextInt, ...)
- hasNext methods for type checking
- nextLine() pitfall and solutions
- Scanner vs IO helpers

Data Parsing

- Integer.parseInt(), Double.parseDouble()
- Handling NumberFormatException
- String cleaning with trim(), toLowerCase()

Input Validation

- **Never trust user input**
- Type, range, format, business rules
- Validate early, fail clearly
- Clear, helpful error messages

Retry Loops

- do-while pattern for guaranteed ask
- Attempt limits for safety
- Multiple validation criteria

Defensive Programming

- Guard clauses for readability
- Input sanitization and validation
- Graceful degradation vs fail fast
- Test with invalid inputs